

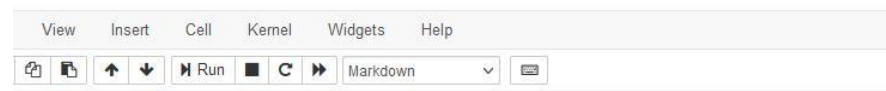


Universitas Gunadarma
PTA 2025/2026

Nama : Nafil Fadillah
NPM : 20123958
Kelas : 3KB02
Mata Kuliah : Praktikum Terapan Teori Graf

Saya akan menjelaskan praktikum yang telah saya lakukan. Teori Graf, adalah suatu teori yang mempelajari sifat suatu grafik, dimana grafik - grafik tersebut terhubung satu sama lain dalam suatu titik. Sebagai contoh penerapannya, seperti di dalam jupyter notebook di bawah ini :

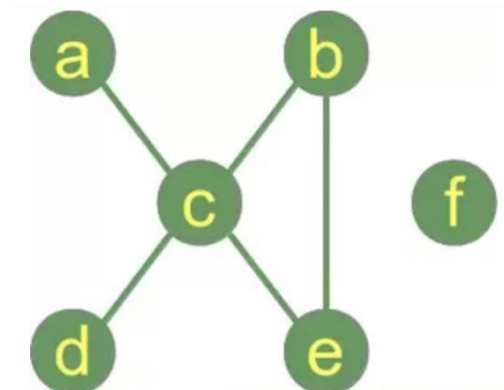
s-On 1 - Teori Graf Last Checkpoint: a minute ago (read only)



Pengenalan Teori Graf

Contoh kasus 1

Dalam contoh kasus pertama kali ini, Kita akan mencoba membangun sebuah graf dengan struktur sebagai berikut.



Untuk membuat representasi bentuk graf seperti berikut, maka kita akan membuat 2 buah fungsi yang dapat membuat representasi tersebut muncul. Sintaksnya seperti di bawah ini

Dalam merepresentasikan bentuk graf tersebut, kita akan membuat 2 fungsi untuk memudahkan

```
In [1]: ## Blok program ini Anda mencoba mendefinisikan node dan hubungan Antara node dalam bentuk obyek.
graph = { "a" : {"c"},
          "b" : {"c", "e"},
          "c" : {"a", "b", "d", "e"},
          "d" : {"c"},
          "e" : {"c", "b"},
          "f" : {}
        }
```

```
In [2]: ## Blok program ini Anda mencoba membuat EDGE dari informasi yang sudah Anda definisikan sebelumnya.
def generate_edges(graph):
    edges = []
    for node in graph:
        for neighbour in graph[node]:
            edges.append([node, neighbour])

    return edges
```

Lalu kita akan mencoba mengeprint graf yang telah kita berikan.

```
In [3]: print(generate_edges(graph))

[['a', 'c'], ['c', 'b'], ['e', 'b'], ['c', 'e'], ['d', 'c'], ['a', 'c'], ['c', 'b'], ['d', 'c'], ['c', 'e'], ['b', 'e']]
```

Kita akan melihat bahwa titik f yang muncul di gambar graf tidak terprint. Itu dikarenakan nilai graf F berada di lokasi yang terisolasi sehingga membuatnya tidak dapat tampil. Agar dapat menampilkan graf F, kita akan memberikan sintaks yang dapat mengetahui dimana nilai f yang terisolasi. Sintaksnya seperti di bawah ini.

```
In [4]: ## Blok program ini Anda mendefinisikan sebuah fungsi untuk mengetahui node mana yang tidak memiliki edge
def find_isolated_nodes(graph):
    """ returns a set of isolated nodes. """
    isolated = set()
    for node in graph:
        if not graph[node]:
            isolated.add(node)
    return isolated
```

```
In [5]: print(find_isolated_nodes(graph))

{'f'}
```

Lalu, di terapan teori graf, kita juga dapat mengetahui informasi yang lengkap tentang graf yang telah kita buat, dengan mendefinisikan class dari graf tersebut

Contoh kasus 2

Dalam contoh kasus kedua kali ini, Kita akan mencoba membangun sebuah graf dan mendapatkan informasi lebih detail mengenai graf tersebut. Untuk memudahkan, Kita akan mendefinisikan sebuah Class dalam bahasa pemrograman python, dimana Class ini berisi beberapa fungsi yang bisa kita gunakan untuk mengetahui informasi detail mengenai stuktur dari graf yang kita miliki

Untuk melakukannya, kita akan memberikan sintaks seperti di bawah ini, dimana kita akan mencoba mendefinisikan class - class dari setiap graf yang ada.

```
In [ ]: """ A Python Class
A simple Python graph class, demonstrating the essential
facts and functionalities of graphs.
"""

class Graph(object):

    def __init__(self, graph_dict=None):
        """ initializes a graph object
        If no dictionary or None is given,
        an empty dictionary will be used
        """
        if graph_dict == None:
            graph_dict = {}
        self._graph_dict = graph_dict

    def edges(self, vertice):
        """ returns a list of all the edges of a vertice"""
        return self._graph_dict[vertice]

    def all_vertices(self):
        """ returns the vertices of a graph as a set """
        return set(self._graph_dict.keys())
```

```

def add_vertex(self, vertex):
    """ If the vertex "vertex" is not in
        self._graph_dict, a key "vertex" with an empty
        list as a value is added to the dictionary.
        Otherwise nothing has to be done.
    """
    if vertex not in self._graph_dict:
        self._graph_dict[vertex] = []

def add_edge(self, edge):
    """ assumes that edge is of type set, tuple or list;
        between two vertices can be multiple edges!
    """
    edge = set(edge)
    vertex1, vertex2 = tuple(edge)
    for x, y in [(vertex1, vertex2), (vertex2, vertex1)]:
        if x in self._graph_dict:
            self._graph_dict[x].add(y)
        else:
            self._graph_dict[x] = [y]

def __generate_edges(self):
    """ A static method generating the edges of the
        graph "graph". Edges are represented as sets
        with one (a loop back to the vertex) or two
        vertices
    """
    edges = []
    for vertex in self._graph_dict:
        for neighbour in self._graph_dict[vertex]:
            if {neighbour, vertex} not in edges:
                edges.append({vertex, neighbour})
    return edges

def __iter__(self):
    self._iter_obj = iter(self._graph_dict)
    return self._iter_obj

def __next__(self):
    """ allows us to iterate over the vertices """
    return next(self._iter_obj)

def __str__(self):
    res = "vertices: "
    for k in self._graph_dict:
        res += str(k) + " "
    res += "\nedges: "
    for edge in self.__generate_edges():
        res += str(edge) + " "
    return res

```

Kemudian, kita akan membuat graf dengan cara mendefinisikan node dan hubungan antar node seperti di kasus pertama

```

In [ ]: g = { "a" : {"d"},
              "b" : {"c"},
              "c" : {"b", "c", "d", "e"},
              "d" : {"a", "c"},
              "e" : {"c"},
              "f" : {}
            }

```

Kemudian, kita mencoba menampilkan informasi terhadap setiap node yang terhubung dengan memberikan sintaks di bawah ini, lalu kita akan mengeprint hasilnya seperti di bawah ini.

```
In [8]: graph = Graph(g)

for vertice in graph:
    print("Edges of vertice {vertice}: ", graph.edges(vertice))

Edges of vertice {vertice}: {'d'}
Edges of vertice {vertice}: {'c'}
Edges of vertice {vertice}: {'e', 'd', 'c', 'b'}
Edges of vertice {vertice}: {'a', 'c'}
Edges of vertice {vertice}: {'c'}
Edges of vertice {vertice}: {}
```

Kemudian, kita akan menampilkan print detail graph yang telah kita tambahkan node baru. Caranya adalah seperti di bawah ini

```
In [9]: graph.add_edge({"ab", "fg"})
graph.add_edge({"xyz", "bla"})

In [10]: print("")
print("Vertices of graph:")
print(graph.all_vertices())

print("Edges of graph:")
print(graph.all_edges())

Vertices of graph:
{'xyz', 'c', 'b', 'bla', 'fg', 'e', 'f', 'a', 'd', 'ab'}
Edges of graph:
[{'d', 'a'}, {'c', 'b'}, {'c', 'e'}, {'d', 'c'}, {'c'}, {'ab', 'fg'}, {'xyz', 'bla'}]
```

Dan terakhir, kita akan mencoba untuk membangun sebuah graf dengan memakai library networkx & matplotlib, dimana kita akan melihat seperti apa bentuk graf yang kita akan buat.

Contoh kasus 3

Dalam contoh kasus ketiga kali ini, Kita akan mencoba membangun sebuah graf dengan menggunakan library networkx dan matplotlib

Yang pertama kita lakukan, adalah mengimport terlebih dahulu library networkx & matplotlib. Jika tidak di import, maka tidak akan bisa membuat bentuk graf tersebut

```
In [ ]: import networkx as nx
import matplotlib.pyplot as plt
```

Selanjutnya, kita akan memberikan sintaks seperti dibawah ini untuk membuat graf

```
In [12]: # Creating a Graph
G = nx.Graph() # Right now G is empty

In [13]: G.add_node(1)
G.add_nodes_from([2,3])

In [14]: G.add_edge(1,2)

In [15]: e = (2,3)
G.add_edge(*e) # * unpacks the tuple
G.add_edges_from([(1,2), (1,3)])

In [16]: G.nodes()
Out[16]: NodeView((1, 2, 3))

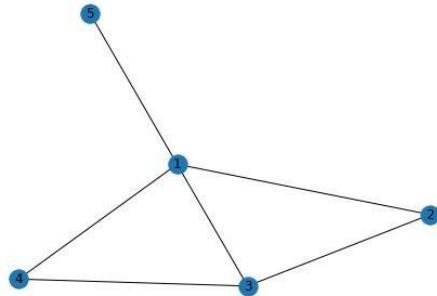
In [17]: G.edges()
Out[17]: EdgeView([(1, 2), (1, 3), (2, 3)])
```

Kita memberikan library nx di graph agar graf tersebut akan memiliki bentuk berupa jaring - jaring. Lalu kita akan memberikan node 1,2 & 3 yang akan dihubungkan satu sama lain

Kemudian, agar kita bisa melihat bentuk graf yang sudah kita buat, maka ketiklah sintaks seperti di bawah ini. Maka kita akan melihat hasilnya seperti di bawah ini

```
In [18]: G.add_edge(1, 2)
G.add_edge(2, 3)
G.add_edge(3, 4)
G.add_edge(1, 4)
G.add_edge(1, 5)
```

```
In [19]: nx.draw(G, with_labels = True)
```



Kemudian, kita akan mencoba tidak menggunakan huruf G besar sebagai variabel dalam membentuk graf

```
In [22]: nx.draw_planar(g, with_labels = True)
```

```
-----
AttributeError                                Traceback (most recent call last)
<ipython-input-22-8c0fc78fd0ca> in <module>
----> 1 nx.draw_planar(g, with_labels = True)

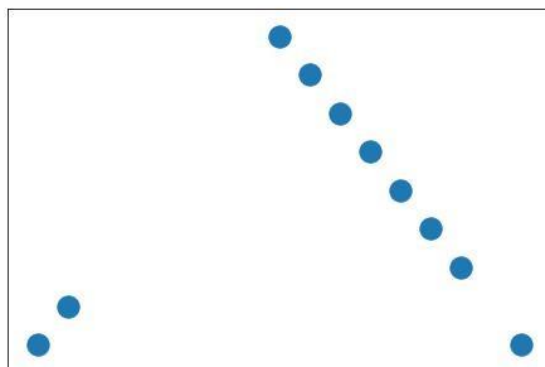
/usr/local/lib/python3.6/dist-packages/networkx/drawing/nx_pylab.py in draw_planar(G, **kwargs)
    1199     function.
    1200     """
-> 1201     draw(G, planar_layout(G), **kwargs)
    1202
    1203

/usr/local/lib/python3.6/dist-packages/networkx/drawing/nx_pylab.py in draw(G, pos, ax, **kwargs)
    121     kwargs["with_labels"] = "labels" in kwargs
    122
-> 123     draw_networkx(G, pos=pos, ax=ax, **kwargs)
    124     ax.set_axis_off()
    125     plt.draw_if_interactive()

/usr/local/lib/python3.6/dist-packages/networkx/drawing/nx_pylab.py in draw_networkx(G, pos, arrows, with_labels, **kwargs)
    334
    335     draw_networkx_nodes(G, pos, **node_kwargs)
-> 336     draw_networkx_edges(G, pos, arrows=arrows, **edge_kwargs)
    337     if with_labels:
```

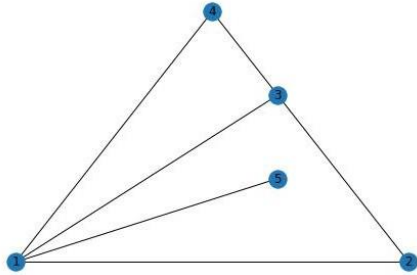
```
--> 640     edgelist = list(G.edges())
    641
    642     if len(edgelist) == 0: # no edges!

AttributeError: 'dict' object has no attribute 'edges'
```



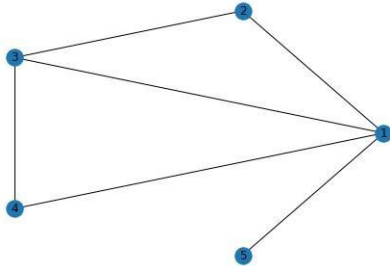
Maka kita akan melihat bahwa grafnya tidak terbentuk dan hanya titik - titiknya saja yang muncul. Ini, disebabkan karena tidak adanya variabel g yang dimasukkan, sehingga akan terjadi error yang menyebabkan graf tidak terbentuk. Sekarang, mari kita ganti dengan G besar.

```
In [29]: nx.draw_planar(G, with_labels = True)
```

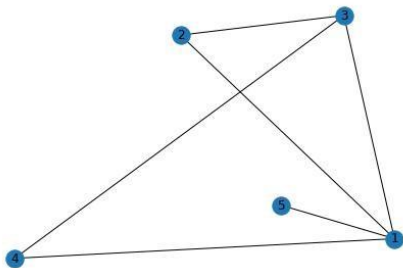


Maka kita akan melihat bahwa jaring - jaring grafnya akan terbentuk. Ini disebabkan, karena variabel G besar itu ada, sehingga jaring - jaring grafnya pun dapat terbentuk. Selanjutnya, kita akan lakukan hal sama terhadap bentuk lain.

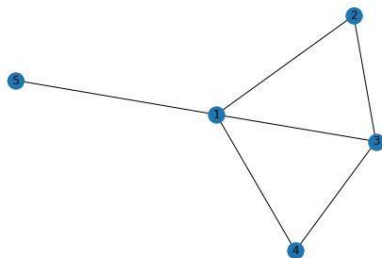
```
In [21]: # drawing in circular layout
nx.draw_circular(G, with_labels = True)
```



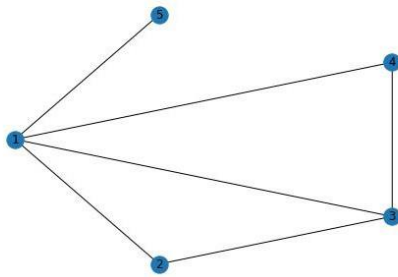
```
In [30]: nx.draw_random(G, with_labels = True)
```



```
In [32]: nx.draw_spring(G, with_labels = True)
```



```
In [33]: nx.draw_shell(G, with_labels = True)
```



Sekarang, coba kita berikan perintah untuk menampilkan gambar graf tanpa networkx.

```
In [20]: plt.savefig("contoh-graf-1.png")
```

```
-----
PermissionError                                Traceback (most recent call last)
<ipython-input-20-62c03e75d183> in <module>
----> 1 plt.savefig("contoh-graf-1.png")

/usr/local/lib/python3.6/dist-packages/matplotlib/pyplot.py in savefig(*args, **kwargs)
    720 def savefig(*args, **kwargs):
    721     fig = gcf()
--> 722     res = fig.savefig(*args, **kwargs)
    723     fig.canvas.draw_idle() # need this if 'transparent=True' to reset colors
    724     return res

/usr/local/lib/python3.6/dist-packages/matplotlib/figure.py in savefig(self, fname, transparent, **kwargs)
    2178     self.patch.set_visible(frameon)
    2179
-> 2180     self.canvas.print_figure(fname, **kwargs)
    2181
    2182     if frameon:

/usr/local/lib/python3.6/dist-packages/matplotlib/backend_bases.py in print_figure(self, filename, dpi, facecolor, edgecolor,
orientation, format, bbox_inches, **kwargs)
    2080         orientation=orientation,
    2081         bbox_inches_restore=_bbox_inches_restore,
-> 2082         **kwargs)

531         _png.write_png(renderer._renderer, fh,
532                         self.figure.dpi, metadata=metadata)

/usr/lib/python3.6/contextlib.py in __enter__(self)
    79     def __enter__(self):
    80         try:
--> 81             return next(self.gen)
    82         except StopIteration:
    83             raise RuntimeError("generator didn't yield") from None

/usr/local/lib/python3.6/dist-packages/matplotlib/cbook/__init__.py in open_file_cm(path_or_file, mode, encoding)
    445 def open_file_cm(path_or_file, mode="r", encoding=None):
    446     r"""Pass through file objects and context-manage `PathLike`'s."""
--> 447     fh, opened = to_filehandle(path_or_file, mode, True, encoding)
    448     if opened:
    449         with fh:

/usr/local/lib/python3.6/dist-packages/matplotlib/cbook/__init__.py in to_filehandle(fname, flag, return_opened, encoding)
    430     fh = bz2.BZ2File(fname, flag)
    431     else:
--> 432         fh = open(fname, flag, encoding=encoding)
    433     opened = True
    434     elif hasattr(fname, 'seek'):

PermissionError: [Errno 13] Permission denied: 'contoh-graf-1.png'
```

Maka, kita akan melihat bahwa sintaksnya error. Ini disebabkan, karena perintah yang diberikan tidak tepat maupun ada, yang membuat kita hanya dapat mengetahui ukuran gambarnya saja seperti di bawah ini.

```
<Figure size 432x288 with 0 Axes>
```